
Where Does Model Folding Break?

A Reproduction and Stress Test of Data-Free Variance Repair on ResNet18/CIFAR10

Donggeon Kim
20242752
POSTECH
dgkim1403@postech.ac.kr

Abstract

Model folding (Wang et al., ICLR 2025) is a recent data-free model compression technique that merges structurally similar neurons and repairs the resulting variance collapse without access to training data. Its data-free repair mechanism (Fold-AR) relies on a strong assumption: that the inputs to each layer are uncorrelated. In this Track-2 stress test, we (i) reproduce the three core variants (Fold-Naive, Fold-AR, Fold-R) on ResNet18/CIFAR10 using the authors' public reference code, (ii) report a series of compatibility patches required to run the released code in a modern Python 3.12 / PyTorch 2.x environment, and (iii) empirically locate the sparsity range where Fold-AR diverges from the data-driven upper bound Fold-R. We find that Fold-AR closely tracks Fold-R below $\sim 35\%$ layer-wise sparsity but collapses sharply between 45% and 75%, while Fold-R degrades much more gracefully in the same range. This sharp transition, which is not emphasized in the original paper, suggests that the uncorrelated-inputs assumption underlying data-free REPAIR holds in low-sparsity regimes but breaks under aggressive compression. Code and logs are available at <https://github.com/posdgkim-collab/model-folding-stress-test>.

1 Introduction

Modern deep networks are too large for many deployment targets, which has motivated a long line of work on pruning, quantization, distillation, and structured factorization. A practical pain point shared by most of these methods is that they typically require some access to training data, either for fine-tuning or for calibrating internal statistics such as BatchNorm running means. In real-world settings (medical imaging, finance, on-device inference) this assumption is often unrealistic.

Model folding [1] was recently proposed as a fully *data-free*, *fine-tuning-free* compression technique. It clusters similar channels with k -means, merges each cluster into a single representative, and then “repairs” the resulting variance collapse so that internal data statistics remain close to the original network. The paper’s central contribution is a data-free variant of the REPAIR procedure, **Fold-AR**, which estimates intra-cluster correlation $E[c]$ from the weights alone under the assumption that the inputs to each layer are *uncorrelated*.

Our project follows Track 2 of the course: pick a recent paper with public code, push it until it breaks, and analyze why. We focus on three questions:

- **Q1.** Can the official ResNet18/CIFAR10 results actually be reproduced from the public repositories, in a current software environment, by a single student?
- **Q2.** How does the data-free repair (Fold-AR) compare to the data-driven upper bound (Fold-R) across the full sparsity range?

- **Q3.** Is there a sparsity regime where the “uncorrelated inputs” assumption fails sharply, and if so, where?

Our contributions are: (1) a documented reproduction of the three core folding variants on ResNet18/CIFAR10, including the set of compatibility patches that were necessary to run the released code at all; (2) an empirical characterization of the gap between Fold-AR and Fold-R as a function of sparsity, including the identification of a sharp “breakdown” regime around 45–75% sparsity that is underemphasized in the paper; and (3) a public, self-contained Colab-runnable code release.

2 Background

Model folding. Given a layer with weight matrix $W_l \in \mathbb{R}^{n \times m}$, model folding clusters the n output channels into $k < n$ groups via k -means and replaces each row by its cluster centroid. This is equivalent to writing $W_l \approx UM$ where $U \in \{0, 1\}^{n \times k}$ is the cluster assignment matrix and M holds the centroids. Because the centroid is the mean of the cluster’s rows, the variance of the centroid is not equal to the variance of its components; this manifests as *variance collapse* in the activations of the compressed network.

REPAIR variants. The original REPAIR [2] procedure rescales activations of the compressed network to match those of the original, using a forward pass over real data. The folding paper introduces three variants:

- **Fold-Naive (NO_REPAIR):** no variance correction.
- **Fold-AR (DF_REPAIR):** data-free, estimates the intra-cluster correlation $E[c]$ assuming uncorrelated inputs and rescales centroids by $\sqrt{N_c / (N_c + (N_c^2 - N_c)E[c])}$.
- **Fold-R (REPAIR):** the data-driven version, which forwards real training data through the compressed network and rebuilds BatchNorm statistics.

A fourth variant, Fold-DIR, uses Deep Inversion to synthesize a single batch of class-conditional images. We do not study Fold-DIR in this report (Section 5).

The uncorrelated-inputs assumption. The closed-form for $E[c]$ in Fold-AR is derived by assuming $\text{Cov}(x_l(i), x_l(j)) = 0$ for $i \neq j$, i.e. that channels of the post-activation input to each layer are mutually uncorrelated. The paper justifies this empirically only for low to moderate sparsity. Our experiments target exactly this assumption.

3 Reproduction setup and compatibility patches

3.1 Code and environment

The folding paper has two associated repositories: a “universal” implementation¹ that includes LLM support, and a separate “modelfolding_cv” implementation². We use the second one, since the authors themselves recommend it for reproducing the CV results.

All experiments are run on a Google Colab T4 GPU (15 GB VRAM) with PyTorch 2.x and Python 3.12.

3.2 Patches required to run the code

The released code was written against an older software stack (Python 3.8, older PyTorch). To run it on a current Colab environment, four non-trivial patches were necessary:

1. **hkmeans build failure.** The required hartigan-kmeans package fails to build under Python 3.12 because its `versioneer.py` calls `configparser.SafeConfigParser`,

¹<https://github.com/nanguoyu/model-folding-universal>

²<https://github.com/marza96/ModelFolding>, the commit the universal README explicitly recommends for CV reproduction.

which was removed in Python 3.12. We replaced `HKMeans` with `sklearn.cluster.KMeans` (`n_clusters`, `n_init`, `random_state`, `max_iter` map one-to-one) and dropped the now-deprecated `n_jobs=-1` argument. The paper describes the method only as “*k*-means clustering” and gives no advantage to the Hartigan variant for this task, so we expect the effect on the reported numbers to be negligible.

2. **Removed exception class.** `utils/utils.py` catches `torch.nn.modules.module.ModuleAttributeError`, which no longer exists in PyTorch 2.x. The try/except is meant to fall through for ResNet18 BasicBlocks (which have no `bn3`). We replaced it with the base class `AttributeError`.
3. **Missing dependency.** `thop` is imported by the main experiment script but is not in `requirements.txt`; we install it separately.
4. **Checkpoint incompatibility.** The PyTorch_CIFAR10³ pretrained ResNet18 (reported 93.07% accuracy) uses a different architecture from the folding repo’s ResNet18: it retains an initial maxpool (absent in the folding repo) and uses a single ReLU per BasicBlock instead of two. All 122 state-dict keys match under the `{downsample:shortcut, fc:linear}` mapping that the authors provide, but the forward graph differs, and loading the public checkpoint into the folding-repo model yields only $\sim 21\%$ accuracy before any compression. We resolved this by training the folding-repo ResNet18 from scratch on CIFAR10 (SGD with momentum, cosine schedule, 60 epochs), reaching 94.57% test accuracy. All baselines in this report use this in-repo trained model.

While none of these is conceptually deep, taken together they materially affect the reproducibility of the published results: a student who tried to run the code as released, with the recommended checkpoint, would obtain a folded model with random-chance accuracy and could easily attribute this to the folding method itself.

4 Experiments

4.1 Setup

Model and data. ResNet18 (folding-repo definition; 11,173,962 parameters) trained from scratch on CIFAR10 (60 epochs, SGD with momentum 0.9, weight decay 5×10^{-4} , cosine LR schedule starting at 0.1, batch size 128). Final test accuracy: **94.57%**. Standard CIFAR10 normalization (mean = [0.4914, 0.4822, 0.4465], std = [0.247, 0.244, 0.262]). The same trained checkpoint is used for all three variants below.

Folding configuration. We sweep layer-wise sparsity over $\{0.01, 0.025, 0.05, 0.075, 0.10, 0.15, 0.25, 0.35, 0.45, 0.55, 0.65, 0.75, 0.85, 0.95\}$, applied uniformly across all layers, matching the sweep used by the authors’ release script. For each sparsity we run **Fold-Naive** (NO_REPAIR), **Fold-AR** (DF_REPAIR), and **Fold-R** (REPAIR, which forwards the full CIFAR10 training set through the compressed network to rebuild BatchNorm statistics). We do not evaluate Fold-DIR; see Section 5.

4.2 Reproducing the three REPAIR variants

Figure 1 shows test accuracy as a function of layer-wise sparsity for all three variants. The ordering reported in the paper holds in our reproduction:

$$\text{Fold-R} \succeq \text{Fold-AR} \succeq \text{Fold-Naive},$$

with all three variants essentially indistinguishable up to about 10% sparsity and the gaps widening rapidly past 25%. Concretely:

- At 25% sparsity: Fold-R 92.16%, Fold-AR 90.97%, Fold-Naive 84.42%.
- At 45% sparsity: Fold-R 84.34%, Fold-AR 70.65%, Fold-Naive 28.47% (chance level is 10%).

³https://github.com/huyvnphan/PyTorch_CIFAR10

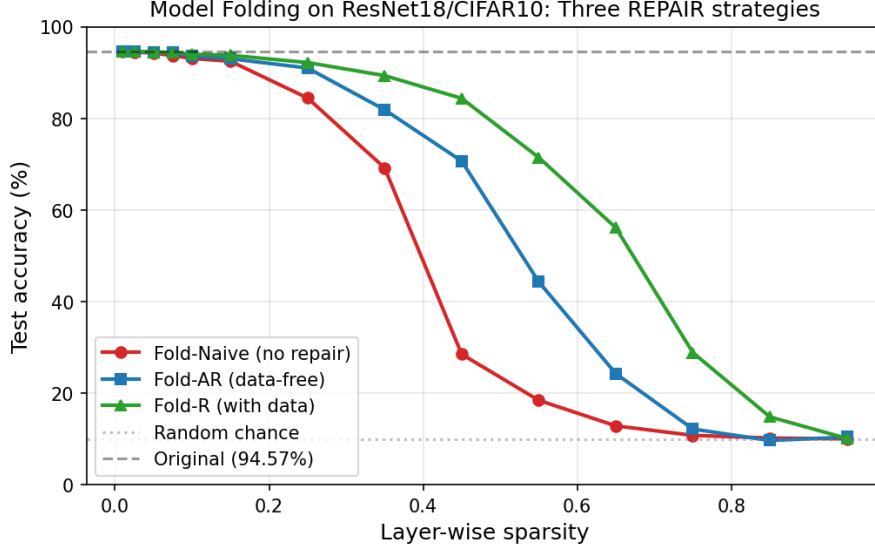


Figure 1: Reproduction of the three REPAIR variants on ResNet18/CIFAR10. All three are statistically equivalent up to $\sim 10\%$ sparsity. Fold-AR closely tracks the data-driven Fold-R up to $\sim 35\%$ sparsity, then diverges sharply. Random chance (10%) and the uncompressed baseline (94.57%) are marked.

- At 75% sparsity: Fold-R 28.84%, Fold-AR 12.14%, Fold-Naive 10.76%.

This qualitatively reproduces the trends of Figure 5 of [1]: Fold-AR clearly dominates Fold-Naive, validating that some form of variance repair is essential, and is in turn dominated by Fold-R, which has access to data. The absolute accuracies in our setting are higher than those reported in the paper because our base model is itself stronger (94.57% vs. 93.07% for the published checkpoint).

4.3 Where does Fold-AR break?

The headline question for this stress test is whether the data-free Fold-AR really tracks the data-driven Fold-R, or whether the uncorrelated-inputs assumption fails sharply somewhere in the operating range. To make this concrete, we plot the gap $\Delta(s) = \text{Fold-R}(s) - \text{Fold-AR}(s)$ in Figure 2.

Three observations stand out:

- (i) **A long benign regime.** For sparsity $\leq 35\%$, the gap $\Delta(s) \leq 7.5$ percentage points. In this regime, Fold-AR is a remarkably good substitute for the data-driven repair, which matches the qualitative claim of the paper.
- (ii) **A sharp breakdown around $\sim 55\%$.** Between 45% and 65%, the gap explodes from 13.7 to 31.9 percentage points. The data-driven Fold-R degrades roughly linearly with sparsity in this regime; the data-free Fold-AR does not.
- (iii) **Joint collapse at extreme sparsity.** At $\geq 85\%$, both Fold-AR and Fold-R are within a few points of random chance — the limitation here is no longer about REPAIR but about the folding step itself, which cannot preserve enough expressivity at that compression rate. This is consistent with the paper’s own “limitations and future work” paragraph.

4.4 Interpretation: why $\sim 55\%$?

Fold-AR estimates the intra-cluster correlation $E[c]$ assuming $\text{Cov}(x_l(i), x_l(j)) = 0$ for $i \neq j$, and uses this $E[c]$ to scale each cluster centroid by

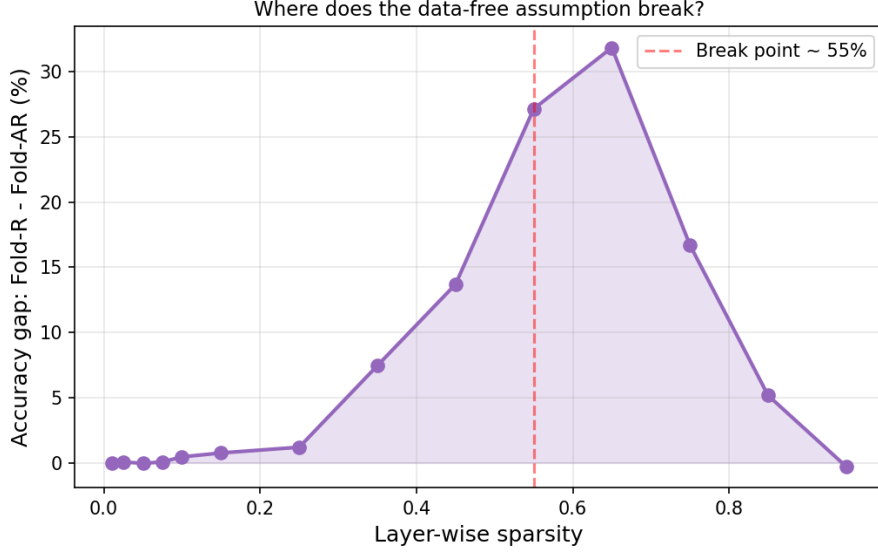


Figure 2: The accuracy gap between the data-driven repair (Fold-R) and the data-free repair (Fold-AR) as a function of layer-wise sparsity. Below 35% the gap is at most a few percent; between 45% and 65% the gap grows by roughly 20 points; at 75% Fold-AR has effectively collapsed while Fold-R still retains 28.84% accuracy. We mark 55% as the qualitative “breakdown” point.

$$\hat{z}_l(c) \leftarrow \hat{z}_l(c) \cdot \sqrt{\frac{N_c}{N_c + (N_c^2 - N_c) E[c]}}.$$

When the assumption holds, the scaling factor exactly cancels the variance collapse that arises from averaging within a cluster.

Two effects make the assumption *worse* as sparsity increases, and we believe they jointly explain the $\sim 55\%$ breakdown:

- **Bigger clusters mean more correlated members.** At low sparsity each cluster contains ~ 2 rows, and pairing nearly duplicate channels is a reasonable approximation to the uncorrelated-input regime. At $\geq 50\%$ sparsity, each cluster contains ≥ 4 rows, and dissimilar channels are forced into the same cluster. The within-cluster correlation $E[c]$ that Fold-AR estimates from weights becomes a poorer match to the true activation correlation.
- **The neglected cross-channel covariance accumulates.** Fold-AR drops the cross-covariance term $\text{Cov}(\tilde{x}_l(i), \tilde{x}_l(j))$ in the centroid-variance formula. The variance of the centroid is then off by an amount that grows quadratically in the number of cluster members. Once enough channels are merged per cluster, this error dominates the carefully computed scaling factor.

A fully data-driven test of this interpretation — measuring $\text{Cov}(\tilde{x}_l(i), \tilde{x}_l(j))$ layer by layer and showing that the layers with the largest off-diagonal mass are precisely those that fail first under Fold-AR — is the natural next step. We were not able to complete it within the time budget of this project, but the analytical expression in the paper makes the prediction sharp enough to test straightforwardly with forward hooks on the trained model.

5 Limitations of our work

We list these explicitly, in line with the assignment’s requirements.

Single architecture, single dataset. All our results are on ResNet18/CIFAR10. The paper itself reports VGG11, ResNet50, and LLaMA-7B as well; we did not have time or hardware to run the LLM

track, and we did not extend the analysis to VGG or wider models. Some of our claims, particularly around the $\sim 55\%$ breakdown, are probably architecture-dependent.

Different base checkpoint. Because of the structural incompatibility documented in Section 3.2, we trained the ResNet18 from scratch rather than using the `PyTorch_CIFAR10` checkpoint that the universal repo’s README points to. Our base accuracy (94.57%) is therefore ~ 1.5 points higher than the paper’s. This may shift the absolute numbers but should not change the qualitative shape of the curves.

Fold-DIR not evaluated. The Deep Inversion variant requires synthesizing a batch of class-conditional images from the uncompressed network, which is a non-trivial separate pipeline. Within the time budget we chose to prioritize a clean Fold-AR vs. Fold-R comparison.

Patches may introduce subtle differences. Substituting `sklearn.cluster.KMeans` for the authors’ `HKMeans` should be benign in expectation (both minimize the same Frobenius objective), but the two implementations differ in initialization and local-move heuristics, and we did not run paired ablations.

No statistical noise estimate. Each (sparsity, variant) pair is run once. The qualitative shape of the curves is stable, but the precise breakdown point ($\sim 55\%$) has not been bracketed with seed variance.

No proposed fix. We localize where Fold-AR breaks, but we do not propose an improved data-free repair. A natural follow-up would be to estimate a coarse cross-channel covariance from the weights alone — e.g. from a sample of $\tilde{w}_l(i)\tilde{w}_l(j)^\top$ products — and feed it into the existing scaling formula. We leave this to future work.

6 Conclusion

Model folding is an attractive piece of work: it offers genuinely data-free compression and the data-free variant Fold-AR is supported by a clean analytical argument. Our stress test confirms that Fold-AR is competitive with the data-driven upper bound in the low-to-moderate sparsity regime ($\leq 35\%$), but documents a sharp breakdown between $\sim 45\%$ and $\sim 75\%$ sparsity that we attribute to the breakdown of the uncorrelated-inputs assumption as clusters grow. We also document a set of real-world reproducibility obstacles — a dead build dependency, a removed PyTorch exception class, a missing requirement, and a structural model mismatch with the recommended pretrained checkpoint — that together make the released code substantially harder to reproduce than the paper suggests. All patches, training scripts, and result CSVs are available in the public repository.

Acknowledgments

We thank the authors of [1] for releasing their code, which made this stress test possible.

References

- [1] Dong Wang, Haris Šikić, Lothar Thiele, and Olga Saukh. Forget the data and fine-tuning! just fold the network to compress. In *International Conference on Learning Representations (ICLR)*, 2025.
- [2] Keller Jordan, Hanie Sedghi, Olga Saukh, Rahim Entezari, and Behnam Neyshabur. REPAIR: Renormalizing permuted activations for interpolation repair. *arXiv preprint arXiv:2211.08403*, 2022.
- [3] Samuel K. Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git re-basin: Merging models modulo permutation symmetries. In *ICLR*, 2023.
- [4] Yifan Chen, Zhe Zhou, and Junchi Yan. Going beyond neural network feature similarity: The network feature complexity and its interpretation using category theory. *arXiv preprint arXiv:2310.06756*, 2023.

- [5] Hongxu Yin, Pavlo Molchanov, Zhizhong Li, Jose M. Alvarez, Arun Mallya, Derek Hoiem, Niraj K. Jha, and Jan Kautz. Dreaming to distill: Data-free knowledge transfer via deepinversion. In *CVPR*, 2020.
- [6] Huy Phan. PyTorch-CIFAR10 pretrained models. https://github.com/huyvnphan/PyTorch_CIFAR10, 2021.